

Glosario - Semana 08: Gestión de Estado con Zustand

A

Action (Acción)

Función dentro del store que modifica el estado. En Zustand, las acciones se definen junto con el estado en el mismo objeto.

```
const useStore = create((set) => ({
  count: 0,
  increment: () => set((state) => ({ count: state.count + 1 })),
}));
```

Atom (Átomo)

Unidad mínima de estado en librerías atómicas como Jotai. Zustand no usa este concepto, pero es útil conocerlo para comparativas.

B

Bear (Oso)

Mascota de Zustand. El nombre "Zustand" significa "estado" en alemán, y la librería usa un oso como logo (de ahí frases como "bear necessities").

Boilerplate

Código repetitivo necesario para configurar una funcionalidad. Zustand se caracteriza por tener muy poco boilerplate comparado con Redux.

C

create()

Función principal de Zustand para crear un store. Retorna un hook personalizado que los componentes usan para acceder al estado.

```
import { create } from 'zustand';
const useStore = create<State>()((set) => ({ ... }));
```

Currying

Técnica donde una función retorna otra función. `create()` en Zustand usa currying para el tipado de TypeScript:

```
create<State>()((set) => ({ ... }));
//           ↑↑ Doble paréntesis
```

D

Derived State (Estado Derivado)

Estado calculado a partir de otro estado. En Zustand se implementa con selectores o funciones dentro del store.

```
getFilteredItems: () => {
  const { items, filter } = get();
```

```
    return items.filter((item) => item.name.includes(filter));  
  };  
};
```

Devtools

Middleware que conecta Zustand con Redux DevTools para debugging visual y time-travel.

```
import { devtools } from 'zustand/middleware';  
create(devtools((set) => ({ ... })));
```

E

Ephemeral State (Estado Efímero)

Estado temporal que no necesita persistirse, como modales abiertos, tooltips, o estados de UI transitorios.

G

get()

Función proporcionada por Zustand para leer el estado actual dentro de acciones.

```
create((set, get) => ({  
  items: [],  
  addItem: (item) => {  
    const current = get().items;  
    set({ items: [...current, item] });  
  },  
}));
```

Global State (Estado Global)

Estado accesible desde cualquier parte de la aplicación, opuesto al estado local de un componente.

H

Hydration (Rehidratación)

Proceso de restaurar el estado desde storage (localStorage/sessionStorage) cuando la aplicación inicia.

Hook

Función de React que permite usar estado y otras características. Zustand crea hooks personalizados para cada store.

I

Immer

Librería que permite escribir código "mutable" que produce actualizaciones inmutables. Zustand tiene middleware nativo para Immer.

```
import { immer } from 'zustand/middleware/immer';
```

Immutability (Inmutabilidad)

Principio donde los datos no se modifican directamente, sino que se crean copias nuevas con los cambios.

L

Listener

Función que se ejecuta cuando el estado cambia. Zustand permite suscribirse a cambios:

```
useStore.subscribe((state) => console.log(state));
```

M

Middleware

Función que envuelve el store para agregar funcionalidad extra (persistencia, devtools, immer).

```
create(
  devtools(
    persist(
      (set) => ({ ... }),
      { name: 'store' }
    )
  )
);
```

Migration (Migración)

Proceso de actualizar datos persistidos cuando la estructura del store cambia entre versiones.

```
persist(fn, {
  version: 2,
  migrate: (persistedState, version) => {
    if (version === 1) {
      // Migrar de v1 a v2
    }
    return persistedState;
  },
});
```

O

onRehydrateStorage

Callback que se ejecuta cuando finaliza la rehidratación del store desde storage.

```
persist(fn, {
  onRehydrateStorage: () => (state) => {
    console.log('Rehidratación completa', state);
  },
});
```

P

Partialize

Opción del middleware persist para guardar solo una parte del estado.

```
persist(fn, {
  partialize: (state) => ({
    items: state.items,
    // No persistir: isLoading, error, etc.
  }),
});
```

Persist

Middleware para guardar el estado en localStorage/sessionStorage automáticamente.

Provider

Componente contenedor necesario en Context/Redux. **Zustand NO requiere Provider**, lo cual simplifica el código.

R

Re-render

Cuando React vuelve a renderizar un componente. Los selectores de Zustand optimizan re-renders.

S

Selector

Función que extrae una parte específica del estado. Optimiza rendimiento evitando re-renders innecesarios.

```
// Selector simple
const count = useStore((state) => state.count);

// Selector con cálculo
const total = useStore((state) =>
  state.items.reduce((sum, item) => sum + item.price, 0),
);
```

set()

Función principal para actualizar el estado en Zustand.

```
set({ count: 10 }); // Reemplazar
set((state) => ({ count: state.count + 1 })); // Actualizar basado en estado anterior
set({ count: 10 }, true); // Reemplazar todo el estado
```

Shallow (Comparación Superficial)

Comparación que revisa solo el primer nivel de un objeto. Usado para optimizar selectores que retornan objetos.

```
import { shallow } from 'zustand/shallow';
const { name, email } = useStore(
  (state) => ({ name: state.name, email: state.email }),
  shallow,
);
```

Slice

Porción del estado con su lógica asociada. Patrón para organizar stores grandes.

```
const createUserSlice = (set) => ({
  user: null,
  setUser: (user) => set({ user }),
});

const createCartSlice = (set) => ({
  items: [],
  addItem: (item) => set((s) => ({ items: [...s.items, item] })),
});
```

Store

Contenedor centralizado del estado de la aplicación en Zustand.

Subscribe

Método para escuchar cambios en el estado fuera de componentes React.

```
const unsubscribe = useStore.subscribe(
  (state) => state.count,
  (count) => console.log('Count cambió a:', count),
);
```

T

Transient Update

Actualización de estado que no persiste ni dispara re-renders, útil para animaciones.

U

useShallow

Hook de Zustand (v4.4+) para comparación superficial automática.

```
import { useShallow } from 'zustand/react/shallow';
const { name, email } = useStore(
  useShallow((state) => ({ name: state.name, email: state.email })),
);
```

V

Vanilla Store

Store de Zustand que funciona fuera de React.

```
import { createStore } from 'zustand/vanilla';
const store = createStore((set) => ({ count: 0 }));
```

Z

Zustand

Librería de gestión de estado para React. Significa "estado" en alemán. Creada por el equipo pmndrs (Poimandres).

 Tabla Comparativa Rápida

Término	Zustand	Redux	Context
Crear store	create()	createStore()	createContext()
Actualizar	set()	dispatch()	setState()
Leer estado	Selector	useSelector()	useContext()
Provider	✗ No necesario	✓ Requerido	✓ Requerido
Middleware	✓ Composable	✓ applyMiddleware	✗ No nativo
DevTools	✓ Middleware	✓ Nativo	✗ No nativo
Bundle size	~1.2KB	~4.5KB	0KB (nativo)

[← Volver a Semana 08](#)